

F1TENTH Race Stack: A Production-Quality ROS 2 Autonomous Racing Architecture, From System Identification to Advanced Head-to-Head Racing

1st Author Name

Department of Robotics and Autonomous Systems
F1TENTH Race Stack Project
email@example.edu

Abstract — This paper documents a complete, production-quality ROS 2 Humble autonomous racing stack developed for the F1TENTH Traxxas Slash 4x4 platform. The system spans the full racing pipeline: hardware system identification, simultaneous localization and mapping (SLAM)-based track mapping, curvature-aware offline racing-line generation, and two complementary onboard controllers — a deterministic Pure Pursuit tracker for Time Trial racing and a CPU-vectorized Model Predictive Path Integral (MPPI) controller for dynamic Head-to-Head racing with obstacle avoidance. A hardware deadman switch enforces operator-in-the-loop safety at all times. The stack is validated both on physical hardware (Intel NUC 12 Pro compute, Hokuyo UST-10LX LiDAR, VESC motor controller) and in the official `f1tenth_gym_ros` 2D physics simulator, enabling development without physical hardware. This paper details the hardware bill of materials, software architecture, node-level algorithmic design, deployment workflow, and live parameter-tuning interface, and contrasts key design decisions against the reference ForzaETH racing stack.

Keywords — autonomous racing, F1TENTH, ROS 2, Model Predictive Path Integral, Pure Pursuit, SLAM, Follow-the-Gap, deadman switch, real-time control.

I. INTRODUCTION

Autonomous racing at 1:10 scale provides a tractable yet representative testbed for algorithms used in full-scale autonomous vehicles, including perception-free reactive navigation, model-based predictive control, and safety-critical human-in-the-loop operation. This paper presents a complete F1TENTH race stack built on ROS 2 Humble that takes a vehicle from an unmapped track to competitive Head-to-Head racing. The stack is organized around four phases: (1) system identification, (2) track mapping, (3) offline racing-line generation, and (4) closed-loop racing using either a Pure Pursuit or MPPI controller. Throughout, a hardware deadman switch guarantees that the vehicle can only move while the operator actively holds a designated joystick button, providing a hard safety guarantee independent of software behavior.

II. HARDWARE PLATFORM

The platform is built on a Traxxas Slash 4x4 chassis. Table I summarizes the bill of materials, including the compute unit, sensing, and actuation hardware used throughout this work.

#	Component	Specification	Notes
1	Chassis	Traxxas Slash 4x4	1:10, 4WD
2	Wheelbase	324 mm	Front-rear
3	Dimensions	296 x 568 mm	W x L
4	Mass	~3.0 kg	Car + elec.
5	Compute	Intel NUC 12 Pro	i7, CPU-only

6	LiDAR	Hokuyo UST-10LX	270°, 40 Hz
7	Motor Driver	VESC MK-VI	ERPM ctrl
8	Joystick	RadioMaster MT12	/dev/input/js0

TABLE I. HARDWARE BILL OF MATERIALS

III. SOFTWARE DEPENDENCIES AND BUILD

A. ROS 2 System Dependencies

The stack targets ROS 2 Humble and depends on the following system packages, installed via apt:

```
sudo apt install -y \
  ros-humble-slam-toolbox \
  ros-humble-joy \
  ros-humble-teleop-twist-joy \
  ros-humble-ackermann-msgs \
  ros-humble-tf2-ros \
  ros-humble-tf2-geometry-msgs \
  ros-humble-rqt-reconfigure \
  ros-humble-rviz2
```

B. Python Dependencies

Offline map and racing-line processing additionally requires the following Python packages:

```
pip install numpy scipy opencv-python \
  matplotlib scikit-image
```

C. Building the Workspace

The package is built using the standard colcon workflow within a ROS 2 workspace. The ROS 2 environment is sourced first, the `f1tenth_race_stack` package is built with `--symlink-install`, the full workspace is built, and the resulting overlay is sourced before any node is launched.

```
# 1. Source ROS 2
source /opt/ros/humble/setup.bash

# 2. Go to your workspace
cd ~/racer_ws

# 3. Build the package
colcon build --symlink-install \
  --packages-select fltenth race stack

# 4. Build the workspace
colcon build --symlink-install \
  --packages-select fltenth race stack

# 5. Source the workspace
source install/setup.bash
```

IV. QUICK REFERENCE COMMANDS (REAL CAR HARDWARE)

This section lists the exact terminal commands used to launch each mode of the race stack on physical hardware. For every command below, Button 5 (SF) on the RadioMaster MT12 joystick must be held for the car to move; releasing it triggers an immediate emergency stop.

A. Mapping (Create Your Track Map)

Manual joystick mapping:

```
ros2 launch fltenth race stack \
  mapping_joystick.launch.py
```

Autonomous mapping (Follow-the-Gap):

```
ros2 launch fltenth race stack \
  mapping_autonomous.launch.py
```

Saving the map (run in a new terminal after mapping):

```
ros2 run nav2 map_server map_saver cli \
  -f ~/racer_ws/src/fltenth_race_stack/\
  maps/fltenth_track
```

B. Path Generation

Generating the optimal racing line from the saved map:

```
ros2 run fltenth race_stack \
  racing_line_generator \
  --ros-args -p map_path:=$HOME/racer_ws/\
  src/fltenth_race_stack/maps/fltenth_track.yaml
```

C. Racing

Time Trials (Pure Pursuit — fastest solo lap):

```
ros2 launch fltenth_race_stack \
  time_trial.launch.py \
  map_path:=$HOME/racer_ws/src/fltenth_race\
  _stack/maps/fltenth_track.yaml \
  racing_line_csv:=$HOME/racer_ws/src/flten\
  th_race_stack/maps/racing_line.csv
```

Head-to-Head (MPPI — dynamic overtaking and obstacle avoidance):

```
ros2 launch fltenth race_stack \
  head_to_head.launch.py \
  map_path:=$HOME/racer_ws/src/fltenth_race\
  stack/maps/fltenth_track.yaml \
  racing_line_csv:=$HOME/racer_ws/src/flten\
  th_race_stack/maps/racing_line.csv
```

V. RUNNING IN SIMULATION

Users without physical F1TENTH hardware can test the entire stack using the official 2D physics simulator. The stack is fully plug-and-play compatible with fltenth_gym_ros.

A. Step 1: Install the Simulator

Run once to download and build the simulator in the workspace:

```
# Install the core Python physics engine
pip3 install git+https://github.com/fltenth/\
  fltenth_gym.git

# Clone the ROS 2 simulator wrapper
cd ~/racer_ws/src
git clone https://github.com/fltenth/\
  fltenth_gym_ros.git

# Build it
cd ~/racer_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install \
  --packages-select fltenth_gym_ros
```

B. Step 2: Run the Simulation

Dedicated sim_ launch files are provided that automatically bypass the hardware deadman switch and disable hardware nodes such as the VESC bridge.

Terminal 1 — start the simulator world:

```
source ~/racer_ws/install/setup.bash
ros2 launch fltenth_gym ros \
  gym_bridge_launch.py
```

Terminal 2 — start the AI brain (Option A, Time Trials with Pure Pursuit):

```
source ~/racer_ws/install/setup.bash
ros2 launch fltenth_race_stack \
  sim_time_trial.launch.py \
  map_path:=$HOME/racer_ws/src/fltenth_race\
  _stack/maps/fltenth_track.yaml \
  racing_line_csv:=$HOME/racer_ws/src/flten\
  th_race_stack/maps/racing_line.csv
```

Terminal 2 — Option B, Head-to-Head with MPPI:

```
ros2 launch fltenth_race_stack \
  sim_head_to_head.launch.py \
  map_path:=$HOME/racer_ws/src/fltenth_race\
  stack/maps/fltenth_track.yaml \
  racing_line_csv:=$HOME/racer_ws/src/flten\
  th_race_stack/maps/racing_line.csv
```

VI. NODE ARCHITECTURE AND ALGORITHM EXPLANATIONS

The stack is organized into controller nodes, mapping and planning nodes, safety and state-management nodes, and supporting utilities. The role and algorithmic behavior of every node is detailed below, together with a contrast against the reference ForzaETH racing stack where applicable. Fig. 1 illustrates the data flow between these nodes, from sensing

through planning, control, the safety gate, and final actuation.

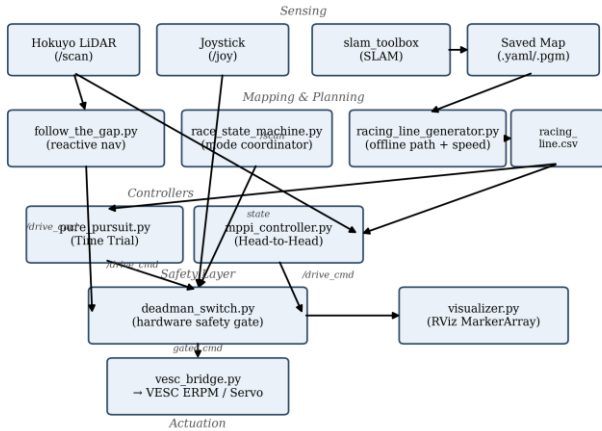


Fig. 1. Node architecture and data flow of the F1TENTH race stack, from sensing through the deadman-switch safety gate to actuation.

A. Controllers (The Brains)

- mppi_controller.py (Model Predictive Path Integral) — Role: used for Head-to-Head Racing. How it works: it uses the CPU to simulate 1,000 different future trajectories (rollouts) in parallel using a dynamic bicycle model. It evaluates each trajectory based on speed, distance to the racing line, and distance to obstacles (LiDAR points). It averages the best trajectories to find an optimal steering and speed command. Because it naturally avoids obstacles, it is well suited for overtaking opponents. Vs. ForzaETH: while ForzaETH often relies on Game-Theoretic NMPC or Lattice Planners for overtaking, which can be very computationally heavy, this stack uses a highly CPU-optimized, vectorized NumPy implementation of MPPI, allowing dynamic overtaking without needing a GPU, running comfortably on an Intel NUC.

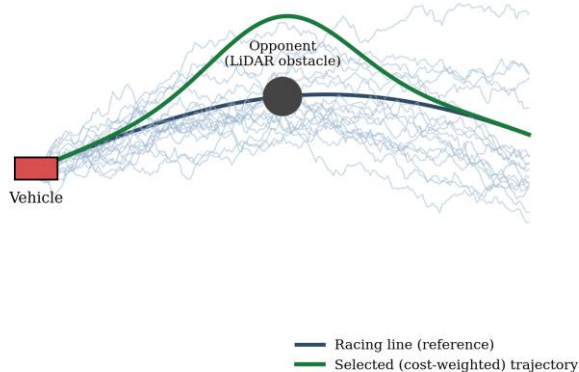


Fig. 2. MPPI samples many candidate rollouts (light blue); the cost-weighted average (green) swerves around the opponent while tracking the reference racing line (navy).

- pure_pursuit.py (Pure Pursuit) — Role: used for Time Trials. How it works: a geometric tracker. It looks ahead a certain distance on the racing line, draws an arc from the car to that point, and calculates the required steering angle. It is extremely fast (runs at 40+ Hz), deterministic, and produces the smoothest, fastest laps when there are no opponents on track. Vs. ForzaETH: ForzaETH typically uses a highly complex Local Trajectory Tracker or NMPC even for time trials. This Pure Pursuit implementation is a deliberate, lightweight alternative; because the racing_line_generator perfectly calculates the speed profile beforehand, a simple geometric tracker is all that is needed to achieve near-optimal time-trial laps with zero computational overhead.

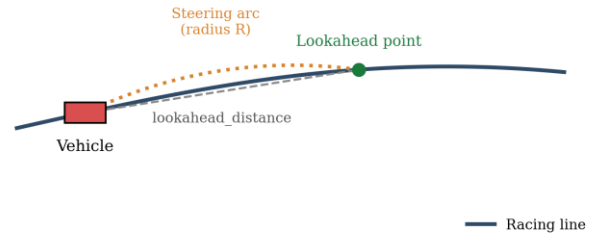


Fig. 3. Pure Pursuit geometry: a lookahead point is selected on the racing line and a constant-curvature steering arc of radius R is computed to reach it.

B. Mapping and Path Planning

- follow_the_gap.py (FTG) — Role: used for Autonomous Mapping. How it works: a purely reactive algorithm. It reads the LiDAR scan, creates a safety bubble around the closest obstacles, finds the largest gap of free space, and steers towards the furthest point in that gap. It scales its speed down during sharp turns. Vs. ForzaETH: this aligns closely with standard F1TENTH reactive architectures, providing a robust fail-safe and an easy way to explore unknown tracks before global optimization is possible.

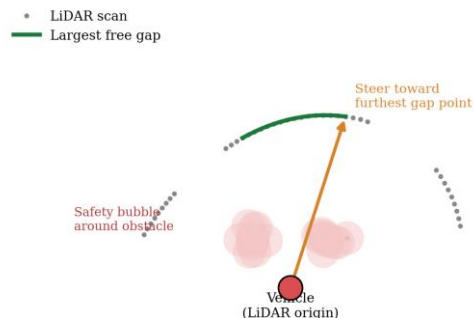


Fig. 4. Follow-the-Gap: safety bubbles are inflated around nearby LiDAR returns, the largest free-space gap is identified, and the vehicle steers toward its furthest point.

- `racing_line_generator.py` — Role: offline path generation and curvature-aware speed profiling. How it works: takes a saved SLAM map (.yaml), uses Euclidean Distance Transforms to find the track boundaries, extracts the centerline, smooths it, and computes an aggressive speed profile based on track curvature. Vs. ForzaETH (Sector Tuning): ForzaETH often uses manual sector tuning, where developers must physically divide the track into sectors (e.g., straightaway, hairpin) and manually assign different target speeds and weights to each sector. This stack automates the process: the generator mathematically evaluates the curvature of every millimeter of the track, automatically assigning high speeds ($v_{\max}$) to straight lines and dynamically ramping down the target speed as a corner sharpens. The controllers therefore automatically brake for corners and accelerate on straights without any manual sector definitions.

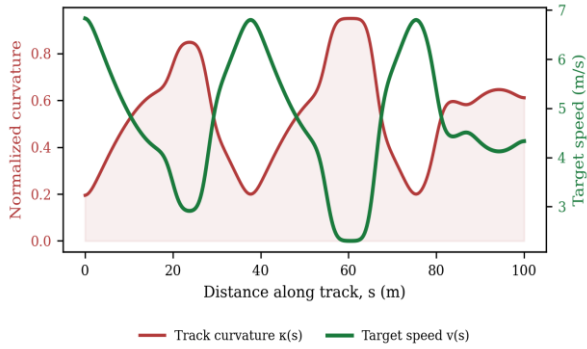


Fig. 6. Curvature-aware speed profiling: as track curvature $\kappa(s)$ rises through a corner, the target speed $v(s)$ automatically ramps down, then back up on the following straight, with no manual sector definitions.

C. Safety and State Management

- `deadman_switch.py` (Hardware Safety Gate) — Role: the ultimate safety layer. How it works: every autonomous planner publishes to `/drive_cmd`. The deadman switch subscribes to this and to the joystick. Unless Button 5 (SF) is held down, it blocks `/drive_cmd` and instead blasts zero-velocity commands to the wheels at 50 Hz. This guarantees the car stops instantly if the operator lets go of the controller.

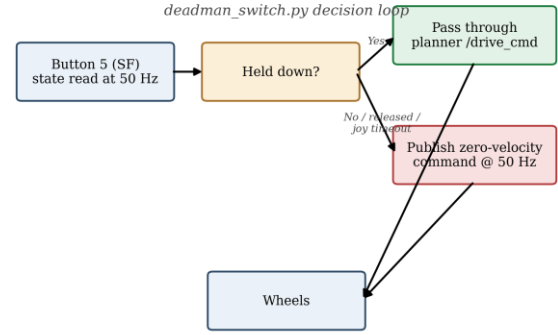


Fig. 5. Decision logic of the deadman switch: planner commands reach the wheels only while Button 5 is held; otherwise zero-velocity commands are published at 50 Hz.

- `race_state_machine.py` — Role: high-level coordinator. How it works: manages states (IDLE, MAPPING_MANUAL, MAPPING_AUTO, TIME_TRIAL, HEAD_TO_HEAD).

D. Utilities

- `vesc_bridge.py` — Converts Ackermann commands (speed, steering) into VESC ERPM and Servo signals.
- `system_id_driver.py` — An open-loop controller used strictly for calibrating the ERPM-to-m/s ratio.
- `visualizer.py` — Publishes rich ROS 2 MarkerArray topics to draw the racing line, velocity arrows, and safety bubbles in RViz.

VII. WORKFLOW: FROM SCRATCH TO ADVANCED RACING

This section follows the exact sequence required to deploy the car on a new track.

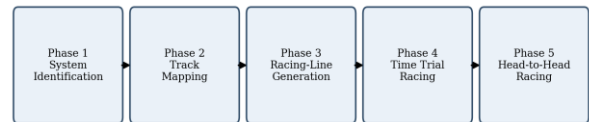


Fig. 7. The five-phase deployment workflow, executed in sequence for every new track.

A. Phase 1: System Identification (First-Time Only)

Calibrate how the motor's RPM relates to real-world speed. Place the car on a 10-meter straight, then launch the ID node:

```
ros2 launch fltenth race_stack \
  system_id.launch.py
```

Look at the terminal output to find the ERPM for a given speed (e.g., 2.0 m/s), and update `erpm_gain` in `config/vesc_bridge_params.yaml`.

B. Phase 2: Build a Track Map

Create a 2D occupancy grid of the track using SLAM.

Option A: Manual Driving (Recommended):

```
ros2 launch fltenth_race_stack \
  mapping_joystick.launch.py
```

- Hold Button 5 (SF) to enable the deadman switch.
- Drive the car around the track using the left (throttle) and right (steering) sticks.
- Drive 2-3 laps to close the loop.

Option B: Autonomous Driving:

```
ros2 launch fltenth_race_stack \
  mapping_autonomous.launch.py
```

- Hold Button 5 (SF). The Follow-the-Gap node will autonomously navigate the track.

Save the map (run in a new terminal):

```
ros2 run nav2_map_server map_saver_cli \
  -f ~/racer_ws/src/fltenth_race_stack/\
  maps/fltenth_track
```

C. Phase 3: Generate the Racing Line

Generate the optimal path and speed profile from the map:

```
ros2 run fltenth_race_stack \
  racing_line_generator \
  --ros-args \
  -p map_path:=$HOME/racer_ws/src/fltenth_\
  race_stack/maps/fltenth_track.yaml \
  -p v_max:=8.0 \
  -p curvature_gain:=2.5
```

This saves racing_line.csv in the maps/ folder.

D. Phase 4: Time Trials (Solo Racing)

For the absolute fastest lap time on an empty track, use Pure Pursuit:

```
ros2 launch fltenth_race_stack \
  time_trial.launch.py \
  map_path:=$HOME/racer_ws/src/fltenth_race_\
  stack/maps/fltenth_track.yaml \
  racing_line_csv:=$HOME/racer_ws/src/flten_\
  th_race_stack/maps/racing_line.csv
```

- Hold Button 5 (SF) to launch the car.
- Tip: tune lookahead_distance if the car is oscillating.

E. Phase 5: Head-to-Head (Racing with Opponents)

When racing against other cars, use MPPI for dynamic overtaking:

```
ros2 launch fltenth_race_stack \
  head_to_head.launch.py \
  map_path:=$HOME/racer_ws/src/fltenth_race_\
  stack/maps/fltenth_track.yaml \
  racing_line_csv:=$HOME/racer_ws/src/flten_\
  th_race_stack/maps/racing_line.csv
```

- Hold Button 5 (SF) to launch. The car will swerve around obstacles.

VIII. DYNAMIC PARAMETER TUNING (LIVE TUNING)

Every single parameter in this stack is live-tunable: nodes do not need to be restarted to change behavior. When any racing mode is launched, a GUI called rqt_reconfigure opens automatically, providing sliders to adjust parameters in real time. Alternatively, the CLI can be used in a new terminal window while the car is running.

A. Tuning Mapping Speeds (Follow-the-Gap and Manual)

```
# Increase the top speed of the autonomous
# FTG mapping
ros2 param set /follow_the_gap \
  ftg.max_speed 4.0

# Change how much FTG slows down for
# sharp corners
ros2 param set /follow_the_gap \
  ftg.min_speed 1.0

# Increase the maximum allowed speed when
# driving manually with the joystick
ros2 param set /deadman_switch \
  manual_max_speed 5.0
```

B. Tuning Time Trials (Pure Pursuit)

```
# Make the car look further ahead
# (smooths out wobbly steering)
ros2 param set /pure_pursuit \
  pure_pursuit.lookahead_distance 1.5

# Scale the speed to 80% of max for a
# safe warmup lap
ros2 param set /pure_pursuit \
  pure_pursuit.speed_scale 0.8
```

C. Tuning Head-to-Head (MPPI)

```
# Make the car more afraid of opponents
# (steers wider around them)
ros2 param set /mppi_controller \
  mppi.weight_obstacle_penalty 1000.0

# Force the car to stick tighter to the
# optimal racing line
ros2 param set /mppi_controller \
  mppi.weight_reference_track 20.0

# Increase computational rollouts
# (better decisions, higher CPU load)
ros2 param set /mppi_controller mppi.K 1500
```

Key default parameter values and descriptions are summarized in Table II.

Parameter	Default	Description
lookahead_distance	1.0 m	Base lookahead
lookahead_gain	0.2	Speed-scaled lookahead
speed_scale	1.0	Racing-line speed mult.
mppi.K	1000	Rollouts simulated

mppi.T	20	Prediction horizon steps
mppi.weight_reference_track	10.0	Off-line penalty
mppi.weight_obstacle_penalty	500.0	LiDAR penalty
mppi.obstacle_clearance_m	0.35 m	Safety radius
deadman_button_idx	5	Deadman index (SF)
manual_max_speed	3.0 m/s	Manual top speed
joy_timeout_s	0.5 s	Disconnect stop

TABLE II. KEY TUNABLE PARAMETERS

IX. DESIGN COMPARISON WITH FORZAETH

Several design choices in this stack diverge deliberately from the ForzaETH reference architecture, favoring CPU-only computation suited to the Intel NUC platform. Where ForzaETH commonly relies on Game-Theoretic NMPC or Lattice Planners for overtaking, which can be very computationally heavy, this stack uses a highly CPU-optimized, vectorized NumPy implementation of MPPI to achieve dynamic obstacle avoidance without needing a GPU, running comfortably on an Intel NUC. Similarly, where ForzaETH typically uses a highly complex Local Trajectory Tracker or NMPC even for time trials, this stack relies on a lightweight Pure Pursuit tracker, made effective because the `racing_line_generator` perfectly calculates the speed profile beforehand, so a simple geometric tracker is all that is needed to achieve near-optimal time-trial laps with zero computational overhead. Finally, in place of ForzaETH's manual Sector Tuning — in which developers must physically divide the track into sectors (e.g., straightaway, hairpin) and manually assign different target speeds and weights to each sector — the racing-line generator in this stack automates the process by mathematically evaluating the curvature of every millimeter of the track, automatically assigning high speeds to straight lines and dynamically ramping down target speed as a corner sharpens, so the controllers automatically brake for corners and accelerate on straights without any manual sector definitions.

X. CONCLUSION

This paper presented a complete, hardware-validated F1TENTH race stack that integrates system identification, SLAM-based mapping, automated curvature-aware racing-line generation, and two complementary racing controllers within a single ROS 2 Humble architecture. By favoring CPU-vectorized MPPI and a precomputed speed profile over GPU-dependent or manually sector-tuned alternatives, the stack achieves competitive Time Trial and Head-to-Head performance on commodity laptop-class compute, while a hardware deadman switch ensures the vehicle remains safe to operate at every stage of development and racing.

REFERENCES

- [1] F1TENTH Foundation, "F1TENTH Autonomous Racing," [Online]. Available: <https://f1tenth.org>
- [2] F1TENTH Gym, "f1tenth_gym_ros: ROS 2 wrapper for the F1TENTH simulator," GitHub repository, 2023.
- [3] ForzaETH, "ForzaETH Race Stack," ETH Zurich, GitHub repository, 2023.
- [4] Open Robotics, "ROS 2 Humble Hawksbill Documentation," 2022.
- [5] G. Williams et al., "Information theoretic MPC for model-based reinforcement learning," in Proc. IEEE ICRA, 2017.